

2_Topics

March 28, 2022

1 Obsługa ROS Topics

1.1 Wprowadzenie - polecenia w terminalu

ROS Topics używane są do komunikacji rozgłoszeniowej. Nie ma znaczenia kto jest nadawcą, a kto odbiorcą wiadomości. Za kodowanie przesyłanej wiadomości odpowiada **Publisher**, a za rozkodowanie **Subscriber**. Typy wiadomości przechowywane są w katalogu srv, a rozszerzenie wiadomości to .msg.

1.1.1 Struktura wiadomości

Z lewej strony należy podać typ wiadomości ROS. Mogą być one bardziej złożone i składać się z już utworzonych wiadomości (ROS msg). Z prawej strony podawana jest nazwa pola.

Typ wiadomości to nazwa paczki + nazwa_wiadomosci.msg. Wyświetlenie przykładowej wiadomości znajdującej się w paczce tsr_materials:

```
[1]: !rosmmsg info pkg_tsr/RobotInfo
```

```
int32 robot_id
string info
```

Parametry z jakimi można wywołać polecenie rosmmsg

```
[2]: # --help - wyświetla pomoc do dowolnego polecenia
!rosmmsg --help
```

rosmmsg is a command-line tool for displaying information about ROS Message types.

Commands:

rosmmsg show	Show message description
rosmmsg info	Alias for rosmmsg show
rosmmsg list	List all messages
rosmmsg md5	Display message md5sum
rosmmsg package	List messages in a package
rosmmsg packages	List packages that contain messages

Type `rosmmsg <command> -h` for more detailed usage

1.1.2 Wyświetlenie listy aktualnie dostępnych topiców

```
[3]: !rostopic list
```

```
/rosout
/rosout_agg
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

1.1.3 Wyświetlenie dostępnej pomocy dla polecenia rostopic

```
[4]: !rostopic --help
```

rostopic is a command-line tool for printing information about ROS Topics.

Commands:

rostopic bw	display bandwidth used by topic
rostopic delay	display delay of topic from timestamp in header
rostopic echo	print messages to screen
rostopic find	find topics by type
rostopic hz	display publishing rate of topic
rostopic info	print information about active topic
rostopic list	list active topics
rostopic pub	publish data to topic
rostopic type	print topic or field type

Type `rostopic <command> -h` for more detailed usage, e.g. '`rostopic echo -h`'

1.1.4 Generowane topic'i przez node'a turtlesim__node

Dla pojedynczego utworzonego robota w przestrzeni nazw na przykładzie turtle1 dostępne są następujące topic'i: - `/turtle1/cmd_vel` - prędkości sterujące robotem - `/turtle1/color_sensor` - kolor - `/turtle1/pose` - położenie robota

Wyświetlenie informacji o topic'u `/turtle1/cmd_vel`:

```
[5]: !rostopic info /turtle1/cmd_vel
```

```
Type: geometry_msgs/Twist
```

Publishers: None

Subscribers:

* /turtlesim (http://localhost:40567/)

1.1.5 Sprawdzenie danych w wiadomości

```
[7]: # po wywołaniu szybko zatrzymać stopem  
# podgląd całej wiadomości  
!rostopic echo -c /turtle1/pose
```

```
x: 5.544444561004639  
y: 5.544444561004639  
theta: 0.0  
linear_velocity: 0.0  
angular_velocity: 0.0  
---  
x: 5.544444561004639  
y: 5.544444561004639  
theta: 0.0  
linear_velocity: 0.0  
angular_velocity: 0.0  
---  
x: 5.544444561004639  
y: 5.544444561004639  
theta: 0.0  
linear_velocity: 0.0  
angular_velocity: 0.0  
---  
^C
```

```
[10]: # podgląd pojedynczego pola  
!rostopic echo -c /turtle1/pose/theta
```

```
0.0  
---  
0.0  
---  
0.0  
---  
^C  
0.0  
---
```

2 Publisher - Python

Podstawową biblioteką do obsługi ROS w Pythonie jest **rospy**. Importowanie wiadomości na podstawie informacji o typie wiadomości jest następujące:

```
import nazwa_paczki.msg import typ_wiadomosci
```

```
[11]: import rospy
      from geometry_msgs.msg import Twist
```

Inicjalizacja node'a, aby ROS mógł jednoznacznie rozpoznać node'a.

Uwaga techniczna. 1 init_node wywoływany w danym zeszycie od Jupyter Notebook.

```
[12]: rospy.init_node("topics_test", anonymous=True)
```

Do utworzenia publishera wykorzystywana jest klasa *Publisher* z biblioteki *rospy*. Przyjmowane kolejno argumenty: - nazwa topic'a (dla już istniejącego w systemie wykorzystywanego przez Subscriber'a lub nowa nazwa) - typ wiadomości, - liczba zakolejkowanych wiadomości.

```
[13]: pub_speed=rospy.Publisher("/turtle1/cmd_vel",Twist,queue_size=10)
```

Utworzenie i uzupełnienie wiadomości.

```
[14]: msg = Twist()
      msg.linear.x = 0.6
      msg.angular.z = 1
```

Do wysłania wiadomości do robota *turtle1* jest metoda klasy *Publisher* o nazwie *publish*, która jako argument przyjmuje typ oczekiwanej wiadomości.

```
[15]: pub_speed.publish(msg)
```

W przypadku wysyłania prędkości wysyłanie wartości prędkości z wysoką częstotliwością spowoduje, że wartości będą się bardzo szybko zmieniały i robot będzie reagował na ostatnio wysłaną wartość. Pojedyncze wysłanie prędkości powoduje, że robot wykonuje ruch z zadaną prędkością około 3s.

Do wykonania przerw pomiędzy kolejnymi ruchami robota można wykorzystać opóźnienie stosując `time.sleep` z biblioteki `time`.

```
import time
```

```
time.sleep(czas_w_sekundach)
```

Zadaniem funkcji jest oczekiwanie określonego czasu przed wykonaniem kolejnej akcji

2.1 Subscriber - Python

```
[16]: !rosservice call reset
```

```
[17]: # funkcja callback wywoływana przez subscibera danych z topicu /informacja. Po
      ↪ każdorazowym
      # pojawieniu się danych funkcja wykonuje się. Jako argument (msg_data)
      ↪ przekazywany jest do
      # funkcji odebrany obiekt typu String (typ wiadomości ROS, a nie
      ↪ programistyczny; inne typy:
      # PoseStamped, PointCloud)
      def callback_function(msg_data):
          print("Subscriber - otrzymana wiadomosc: ", msg_data.data)
```

Do utworzenia subscibera wykorzystywana jest klasa Publisher z biblioteki rospy. Przyjmowane kolejno argumenty: - nazwa topic'u - typ wiadomości - nazwa funkcji, która jest wywoływana do odebrania danych z odczytanej wiadomości

```
[18]: from std_msgs.msg import String
      my_subscriber = rospy.Subscriber("informacja",String,callback_function)
```

Subscriber - otrzymana wiadomosc: to jest przykładowa wiadomosc

Wyłączenie subscriber'a.

```
[ ]: my_subscriber.unregister()
```

2.2 Jednoczesny Publisher Subscriber - przykład

```
[ ]: import rospy
      from turtlesim.msg import Pose
      from geometry_msgs.msg import Twist
      rospy.init_node("topics_test2", anonymous=True)

      vel_topic_name = "/turtle1/cmd_vel" # UZUPEŁNIĆ
      pub_velocity = rospy.Publisher(vel_topic_name,Twist,queue_size=10)

      direction_right = True
      def robot_control(message):
          """Analiza wiadomości i wysłanie jej na innym topicu"""
          global direction_right
          vel_msg = Twist()
          if direction_right:
              vel_msg.linear.x = 0.5
              vel_msg.angular.z = 0
```

```

else:
    vel_msg.linear.x = -0.5
    vel_msg.angular.z = 0

if message.x > 7:
    direction_right = False
elif message.x < 2:
    direction_right = True
# wysłanie przeanalizowanych danych
pub_velocity.publish(vel_msg)

# odebrać wiadomość z topicu /
pose_topic_name = "/turtle1/pose" # UZUPEŁNIĆ
subscriber= rospy.Subscriber(pose_topic_name, Pose, robot_control)

```

[]: